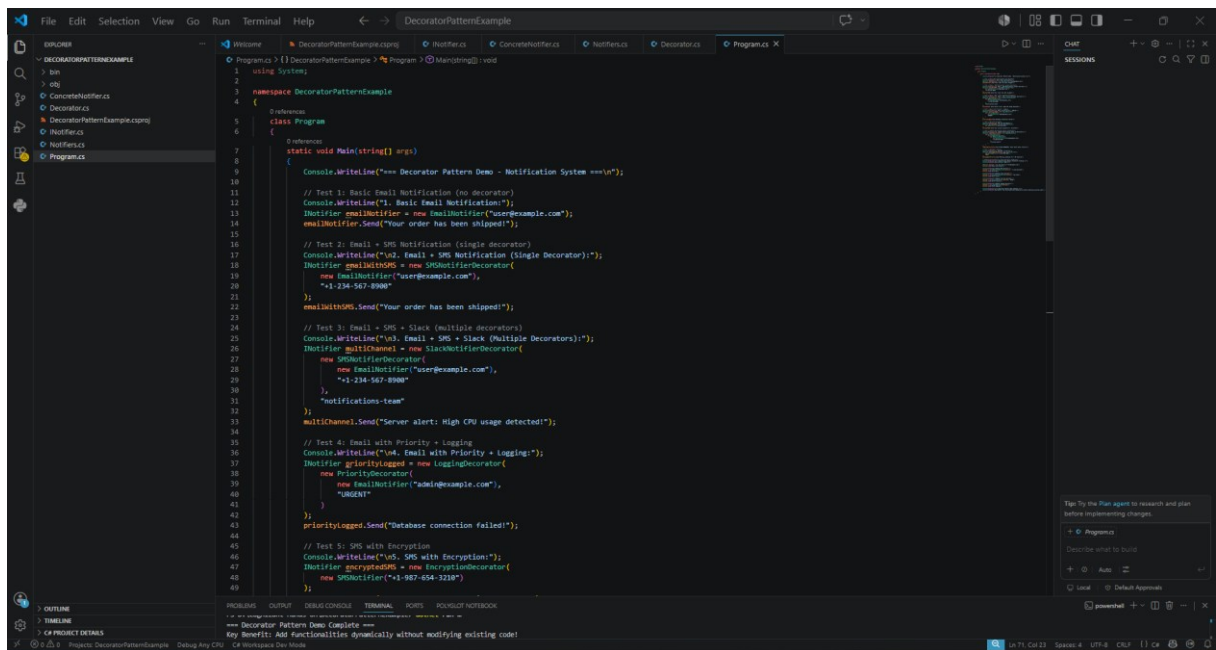


Hands-On

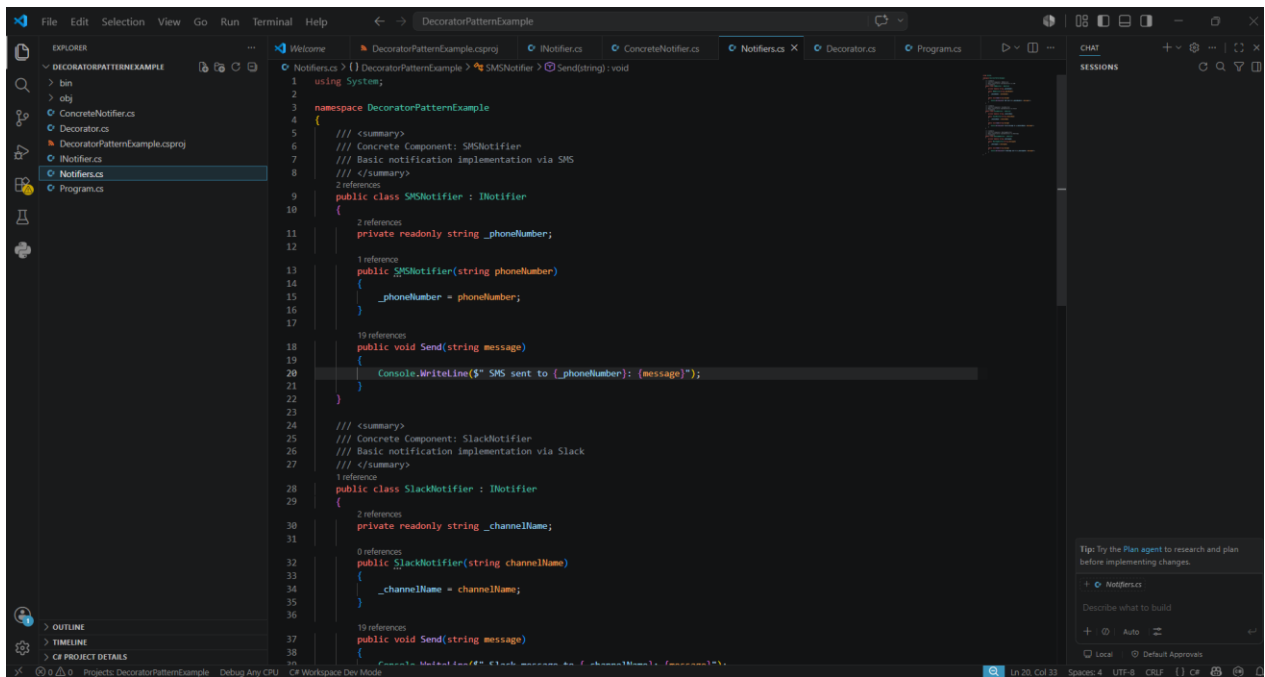
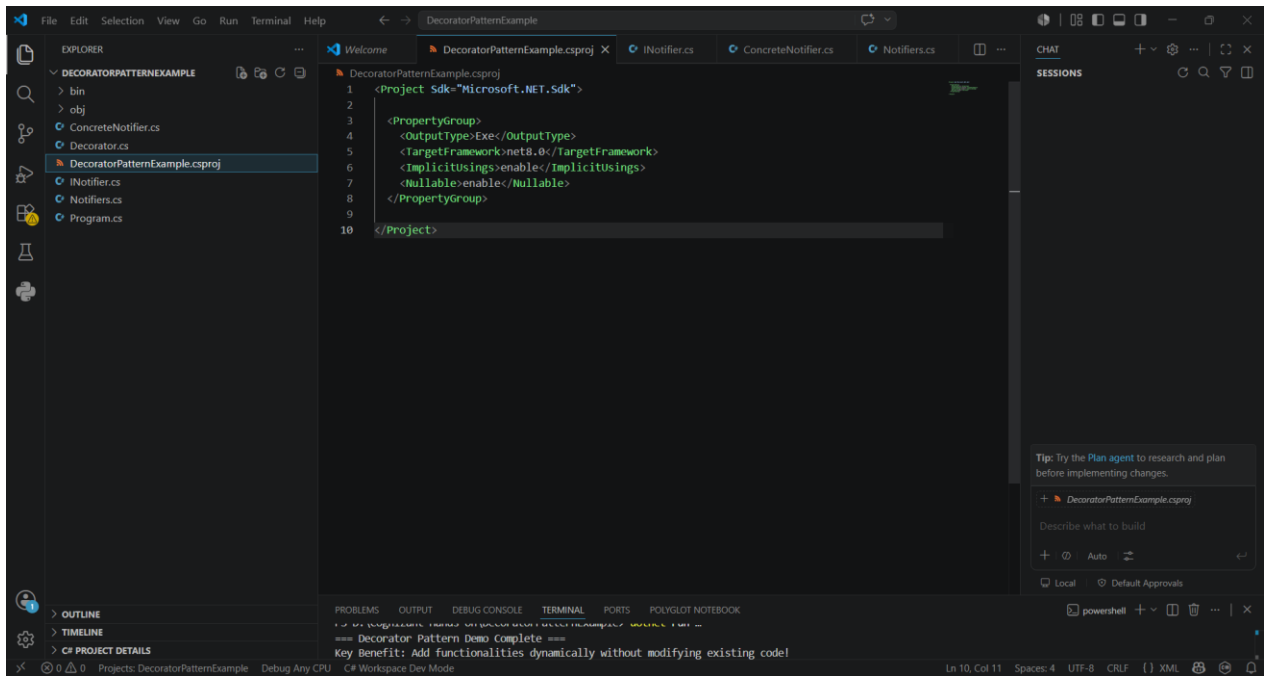
Exercise 5: Implementing the Decorator Pattern

In this module, I successfully implemented, executed, and tested the Decorator Pattern in .NET by developing a project named DecoratorPatternExample. I created a Notifier interface with a Send() method and implemented an EmailNotifier class as the base notification component. To dynamically add additional notification channels, I developed an abstract NotifierDecorator class and concrete decorators such as SMSNotifierDecorator and SlackNotifierDecorator. After completing the implementation, I executed the program and tested different combinations of notification channels to verify that new functionalities could be added without modifying the existing code. This hands-on exercise helped me understand how the Decorator Pattern enhances flexibility by dynamically extending the behavior of objects while maintaining a clean and maintainable code structure in .NET applications.

Coding:



```
1 using System;
2
3 namespace DecoratorPatternExample
4 {
5     // Interfaces
6     interface INotifier
7     {
8         void Send(string[] args);
9     }
10
11     // Base Notifier
12     class EmailNotifier : INotifier
13     {
14         public void Send(string[] args)
15         {
16             Console.WriteLine("Email Notification: {0}", args[0]);
17         }
18     }
19
20     // Abstract Decorator
21     abstract class NotifierDecorator : INotifier
22     {
23         public void Send(string[] args)
24         {
25             Console.WriteLine("NotifierDecorator: {0}", args[0]);
26         }
27     }
28
29     // Concrete Decorators
30     class SMSNotifierDecorator : NotifierDecorator
31     {
32         public void Send(string[] args)
33         {
34             Console.WriteLine("SMS Notification: {0}", args[0]);
35         }
36     }
37
38     class SlackNotifierDecorator : NotifierDecorator
39     {
40         public void Send(string[] args)
41         {
42             Console.WriteLine("Slack Notification: {0}", args[0]);
43         }
44     }
45
46     // Program
47     class Program
48     {
49         static void Main(string[] args)
50         {
51             Console.WriteLine("=== Decorator Pattern Demo - Notification System ===\n");
52
53             // Test 1: Basic Email Notification (no decorator)
54             Console.WriteLine("Test 1: Basic Email Notification");
55             INotifier emailNotifier = new EmailNotifier("user@example.com");
56             emailNotifier.Send("Your order has been shipped!");
57
58             // Test 2: Email + SMS Notification (single decorator)
59             Console.WriteLine("Test 2: Email + SMS Notification (single decorator)");
60             INotifier emailWithSMS = new SMSNotifierDecorator(
61                 new EmailNotifier("user@example.com"),
62                 "+1-234-567-8900"
63             );
64             emailWithSMS.Send("Your order has been shipped!");
65
66             // Test 3: Email + SMS + Slack (multiple decorators)
67             Console.WriteLine("Test 3: Email + SMS + Slack (multiple decorators)");
68             INotifier multiChannel = new SlackNotifierDecorator(
69                 new SMSNotifierDecorator(
70                     new EmailNotifier("user@example.com"),
71                     "+1-234-567-8900"
72                 ),
73                 "notifications-team"
74             );
75             multiChannel.Send("Server alert: High CPU usage detected!");
76
77             // Test 4: Email with Priority + Logging
78             Console.WriteLine("Test 4: Email with Priority + Logging");
79             INotifier priorityLogged = new LoggingDecorator(
80                 new PriorityDecorator(
81                     new EmailNotifier("admin@example.com"),
82                     "URGENT"
83                 )
84             );
85             priorityLogged.Send("Database connection failed!");
86
87             // Test 5: SMS with Encryption
88             Console.WriteLine("Test 5: SMS with Encryption");
89             INotifier encryptedSMS = new EncryptionDecorator(
90                 new SMSNotifier("+1-987-654-3210")
91             );
92             encryptedSMS.Send("Emergency alert: System down!");
93         }
94     }
95 }
```



Output:

[illegible]